

AI-Based Fake Identity and Document Screening System - Action Plan

- [AI-Based Fake Identity & Document Screening System — Action Plan](#)
 - [1. Core Positioning \(say this in every pitch/Q&A\)](#)
 - [2. Total Module Count: 6](#)
 - [3. System Architecture \(data flow\)](#)
 - [4. Module Composition — Tech Stack, Prerequisites, and Edge Cases](#)
 - [5. Risk Scoring Engine — Two-Dimensional Design](#)
 - [6. Fraud Scenario → Module Responsibility Matrix](#)
 - [7. Prototype vs. Real-World Deployment — What Changes, What Stays the Same](#)
 - [8. Data Plan](#)
 - [9. 8-9 Day Sprint Plan](#)
 - [9a. Mock-First Integration Strategy — Why Day 2-3, Not Day 7](#)
 - [9b. Standardized Mock Output — Exact Stub Return Value Per Module](#)
 - [10. Do's and Don'ts](#)
 - [11. Multi-Document-Type Handling — Config-Driven Strategy](#)
 - [12. Data & Mock DB Task Allocation — By Skill Level](#)
 - [13. Module I/O Contracts — Avoiding Integration and Explainability Issues](#)
 - [14. Team Coordination Guidelines](#)
 - [15. Overall Problem Statement Coverage — What's Addressed, What's Not](#)

AI-Based Fake Identity & Document Screening System — Action Plan

Team size: 2-3 people | **Timeline:** 8-9 days | **Theme:** Blockchain & Cybersecurity

1. Core Positioning (say this in every pitch/Q&A)

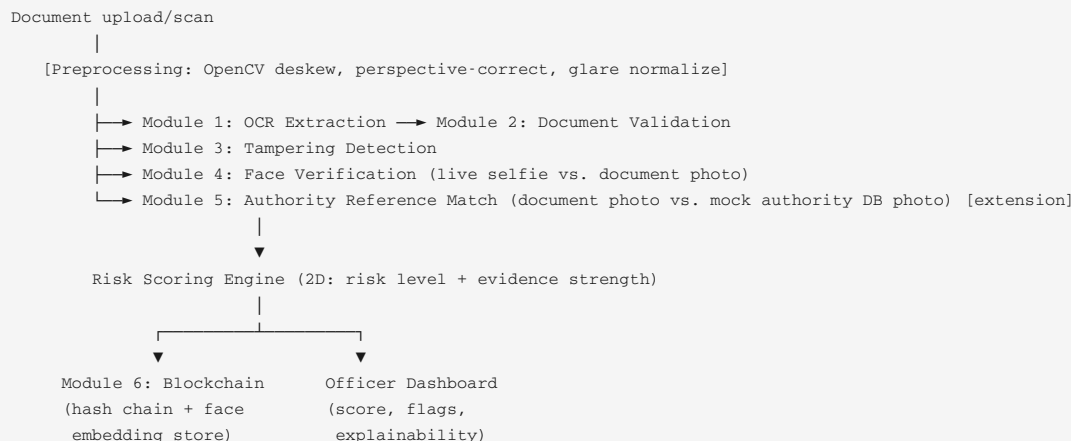
“This is a decision-support triage system that assists border officers, not an autonomous gatekeeper. It automates the digitally-verifiable checks so officers focus their time and physical inspection tools (UV, hologram, RFID) only on the smaller set of cases that genuinely need it — and every decision, including any later found wrong, is permanently and identifiably logged.”

2. Total Module Count: 6

#	Module	Status
1	OCR Extraction	Core — must-have
2	Document Validation	Core — must-have
3	Tampering Detection	Core — must-have (this is the theme’s “core AI innovation”)
4	Face Verification	Core — must-have
5	Authority Reference Match (extension)	Stretch — build only after 1-4 are solid
6	Blockchain Audit Layer	Core — required by theme, cuts across all above

Modules 1-4 are exactly what the problem statement lists. Module 5 is your own addition, built specifically to close the “flawless forgery” gap identified during our discussion. Module 6 is not a screening module — it’s a logging/integrity layer that every other module writes to.

3. System Architecture (data flow)



4. Module Composition — Tech Stack, Prerequisites, and Edge Cases

Each module below lists: **what it does** → **tech stack** → **what data/DB it needs before it can be tested** → **which specific edge cases from our discussion it is responsible for catching**.

Module 1 — OCR Extraction

- **Function:** Convert document image into structured field data (name, passport no., DOB, expiry, nationality, gender, visa fields).
- **Tech stack:** PassportEye / `mrz` library (MRZ zone) + PaddleOCR/EasyOCR (visual zone) + OpenCV (preprocessing: deskew, crop, contrast).
- **Prerequisites:**
 - MIDV-500 image subset (5-8 doc types, `images/` folders)
 - MIDV-500 ground truth JSON (for accuracy benchmarking — not required for the module to run, only to measure it)
 - No database required
- **Edge cases owned:**
 - Output must include a **per-field OCR confidence score** — this feeds the evidence-strength axis later (blurry/low-quality capture must lower confidence, not silently pass)
 - Must correctly parse MRZ even from imperfect phone-photo captures (tested via preprocessing step)

Module 2 — Document Validation

- **Function:** Verify extracted data against format standards, checksum math, and known-record status.
- **Tech stack:** Pure Python (regex, `datetime`) — rule-based, no ML.
- **Prerequisites:**
 - Output of Module 1 (depends on it directly — cannot run standalone)
 - **Mock DB #1 — Issuance/Blacklist table:** a small local table of passport/visa numbers marked issued / expired / blacklisted. You create this yourself (5-10 entries is enough, some drawn from your MIDV-500 subset, some invented).
 - ISO 3166 country code list (free public reference table, one-time download)
- **Edge cases owned:**
 - MRZ checksum verification (7-3-1 weighted formula) — catches physical alteration where MRZ wasn’t correctly recomputed
 - **Visible printed text vs. MRZ text cross-check** — catches physical alteration where one zone was updated and the other wasn’t

- Expired/blacklisted document — catches a genuinely clean document that's simply invalid per records
- This is the module that catches "checksum correct but data doesn't match records" cases

Module 3 — Tampering Detection (core AI innovation)

- **Function:** Detect digital and (partially) physical tampering signs in the image itself.
- **Tech stack:** Pillow (ELA, EXIF/metadata read), OpenCV + SIFT/ORB (copy-move detection via keypoint self-matching).
- **Prerequisites:**
 - Original uploaded/scanned image (works standalone, doesn't depend on Modules 1/2)
 - **DIY tampered fixture set:** 3-5 documents you deliberately edit (photo swap, digit alteration, stamp clone) — required to have any positive test case at all; without this, the module has nothing to prove it works against
 - **DIY "single-save clean" fixture set:** documents built once via Python + Pillow (not edited in GIMP/Photoshop) — required so you have a genuine negative test case that doesn't falsely trigger ELA
- **Edge cases owned:**
 - Digital file edited pre-upload (ELA + EXIF) — this module's strongest, most reliable catch
 - Same-image cloning, e.g. duplicated stamp (copy-move/SIFT) — the case ELA alone misses
 - Photo boundary/edge artifacts from a physically swapped photo, poorly finished — weak but real signal
 - **Evidence-strength tracking:** ELA signal clarity (not just error magnitude) must be output separately, so a deliberately degraded/recompressed image is flagged as "inconclusive," not "clean"
 - **Known limitation this module does NOT close:** a cleanly executed physical forgery (correct paper, correct print, no digital editing at all) — no digital forensic signal exists to catch this; state this explicitly, do not claim otherwise

Module 4 — Face Verification

- **Function:** Confirm the live person presenting the document matches the document's own photo.
- **Tech stack:** DeepFace or `face_recognition` (pretrained FaceNet/ArcFace embeddings), cosine similarity.
- **Prerequisites:**
 - **DIY face-match pairs (5-6):** MIDV-500 template + team member's photo pasted in (built via Pillow, single save) + that member's live selfie
 - **DIY face-mismatch pairs (3-4):** original unedited MIDV-500 photo + a team member's live selfie
 - No database required to run — but you need your own threshold, tuned against the pairs above
- **Edge cases owned:**
 - Identity impersonation using someone else's document — mismatch caught here even without detecting the tampering itself
 - Photo swapped to a *different, wrong* person — caught here
 - **Known limitation this module does NOT close:** photo swapped to the *forger's own* face, done cleanly — will PASS here, because the live person genuinely matches the (swapped) document photo. This is the exact gap Module 5 exists to close.

Module 5 — Authority Reference Match (extension module, build only if time allows)

- **Function:** Compare the document's own photo against an independent, authority-registered reference photo — closing the flawless-forgery gap where Module 4 alone would pass.
- **Tech stack:** Same face-embedding library as Module 4 (DeepFace/ `face_recognition`) — no new tech required, just a second comparison against a different reference image.
- **Prerequisites:**
 - **Mock DB #2 — Authority Reference Photo table:** a small local table mapping passport number → a "reference photo" you designate as the authority-issued original (e.g., the original unedited MIDV-500 photo for that document, before you swapped it for your tampered fixture)
 - Depends on Module 1's extracted passport number to look up the correct reference record
- **Edge cases owned:**
 - **The flawless physical forgery case:** correct checksum, correct MRZ, forger's own face swapped in cleanly, valid/unflagged doc number. Module 4 passes this (live face matches swapped photo); Module 5 catches it, because the document's *current* photo no longer matches the *authority's original* record for that passport number — a mismatch a forger cannot fix without access to the authority's database.
- **Build note:** This only needs a mock lookup table and one extra embedding comparison — cheap to add once Modules 1-4 are stable. If you run out of time, do not skip presenting it — describe it in your pitch as a designed extension (see Section 7).

Module 6 — Blockchain Audit Layer

- **Function:** Two distinct jobs, don't conflate them — (a) make your own stored verification records tamper-evident, (b) detect the same face/document reappearing under a different identity across checkpoints.
- **Tech stack:** Python `hashlib` (SHA-256 hash chain). Stretch: `web3.py` + local Ganache testnet for a “real blockchain” demo look.
- **Prerequisites:**
 - Output of the Risk Scoring Engine (writes after every screening — depends on all other modules having run)
 - Storage for the ledger itself (a local file or lightweight DB is enough — append-only)
- **Edge cases owned:**
 - **(a) Insider/system-level record tampering:** hash chain `H_n = SHA256(H_{n-1} || record_data)` — altering any past record breaks every hash after it. This protects your database, not the original document.
 - **(b) Multiple identities / same person reused across checkpoints:** separate mechanism — nearest-neighbor similarity search of each new face embedding against all previously stored embeddings in the ledger. A close match under a different declared name is your flag. This is what actually catches identity reuse; the hash chain does not do this by itself.

5. Risk Scoring Engine — Two-Dimensional Design

Combines outputs of Modules 2, 3, 4 (and 5, if built). Don't output binary safe/unsafe.

Axis 1 — Risk score (rule violations + tamper/mismatch signals from Modules 2, 3, 4, 5) **Axis 2 — Evidence strength** (OCR confidence from Module 1, ELA signal clarity from Module 3, face-match margin from Module 4/5, doc-type familiarity)

	High evidence strength	Low evidence strength
Low risk score	Clear	Medium — request rescan/manual look
High risk score	Escalate	Escalate (worse, not better)

6. Fraud Scenario → Module Responsibility Matrix

Scenario	Module(s) that catch it	Notes
Digital file edited pre-upload	Module 3 (ELA/EXIF/copy-move)	Strongest category
Physical alter, MRZ not recomputed	Module 2 (checksum + text/MRZ cross-check)	
Physical photo swap, visible edge/lamination flaw	Module 3 (copy-move/edge artifacts)	Weak but real
Photo swap to a different, wrong person	Module 4 (face verification)	
Genuine document but blacklisted/expired	Module 2 (mock DB #1)	Independent of image analysis
Photo-of-screen/photocopy submission	Module 3 (recapture artifacts — stretch)	
Same face/document reused under different identity across checkpoints	Module 6b (embedding similarity search)	Not the hash chain
Insider alters a stored past record	Module 6a (hash chain)	Protects ledger, not the document
Flawless physical forgery: correct checksum, correct MRZ, forger's own face swapped in cleanly, valid/unflagged doc number	Module 5 (authority reference match), if built	Without Module 5: not caught at first contact — this is the honest theoretical ceiling of Modules 1-4 alone

7. Prototype vs. Real-World Deployment — What Changes, What Stays the Same

Use this table directly when judges ask “is this really deployable.”

Component	In this prototype	In a real deployment	What stays the same
Module 2 issuance/blacklist check	Mock local table (Mock DB #1)	Live connection to actual government passport/visa issuance systems	The validation logic and rule structure are identical

Module 5 authority reference photo	Mock local table (Mock DB #2)	RFID/NFC chip on the e-passport, storing a government-signed original photo, read independently at the border	The comparison logic (document photo vs. independent reference) is identical — only the source of the reference photo changes
Physical forgery detection (paper, ink, holograms)	Not covered — no hardware access	UV/IR light scanners, hologram verifiers used alongside this system by the officer	This system was always designed as a first-line triage layer, not a replacement for these tools — this doesn't change
Blockchain ledger	Single local hash-chain, one machine	Distributed ledger shared and synced across multiple checkpoints, likely with real consensus/network infrastructure	The hash-chain integrity principle and embedding-search logic are identical — only the network/scale changes
Document capture	Uploaded image files (scan or photo)	Dedicated document scanners at checkpoint kiosks (preferred) with phone/photo capture as fallback	The OpenCV preprocessing step is designed for both, so no change needed
Face embeddings	Pretrained DeepFace/ <code>face_recognition</code> , no fine-tuning	Same — pretrained embeddings are already production-grade; likely just a larger/more diverse verification threshold tuning pass	Core approach is identical

One-line summary for judges: *“Everywhere we used a mock database, we mocked it because the data is government-restricted or requires infrastructure we don’t have — not because the underlying design is incomplete. The comparison logic, scoring formulas, and detection algorithms are the same ones a production system would run; only the data source behind each lookup changes.”*

8. Data Plan

Base clean documents: MIDV-500 dataset (public, synthetic, ICAO-format-correct — 50 doc types, `images/` + `ground_truth/` JSON per folder). Use ground truth JSON as your OCR accuracy benchmark.

What you build yourself: | Fixture | How to build | Feeds which module(s) | |—|—| | Tampered documents (3-5) | Edit MIDV-500 JPEGs in GIMP: swap photo, alter DOB digit, clone stamp. Save as JPEG | Module 3 | | Single-save “clean” docs | Build via Python + Pillow, one save, no editing software | Module 3 (clean baseline), Module 4 (match case base) | | Face match pairs (5-6) | Single-save doc with team member’s photo + that member’s live selfie | Module 4 | | Face mismatch pairs (3-4) | Original unedited MIDV-500 photo + a team member’s live selfie | Module 4 | | Mock DB #1 entries | Hand-pick MIDV-500 passport numbers, mark issued/expired/blacklisted | Module 2 | | Mock DB #2 entries | Map passport number → original unedited MIDV-500 photo as the “authority reference” | Module 5 |

Total target: ~25-35 documents + ~10 face pairs + two small mock DB tables. This is test fixtures, not training data — no module in this system is trained.

9. 8-9 Day Sprint Plan

Day	Focus	Modules	Owner split
1	Stack finalized, MIDV-500 subset downloaded, repo + FastAPI skeleton, shared I/O schema (Section 13) agreed and written down	Setup	All
2	Every module owner writes a stub function returning hardcoded mock output matching the shared schema (e.g. Module 1’s stub returns a fixed fake JSON, no real OCR yet)	Mock stubs	All, in parallel
2-3	Backend wires all mock stubs together end-to-end — upload → fake OCR → fake validation → fake tamper score → fake face score → fake risk score → fake ledger write → dashboard displays it	Full pipeline (mocked)	Whoever owns backend/integration
3 onward	Real module logic replaces each stub one at a time , in	Module 1 (real)	Person A

	place, without changing its output schema — MRZ parsing + visual-zone OCR → structured JSON with confidence scores		
3 onward	Checksum, expiry, format rules, cross-check, Mock DB #1	Module 2 (real)	Person A
3-4	ELA + EXIF + copy-move (SIFT/ORB)	Module 3 (real)	Person B
3-4	Face detection + crop + similarity	Module 4 (real)	Person B/C
4	Build all DIY fixtures: tampered docs, single-save clean docs, face pairs, Mock DB #1 + #2 entries	Data (Section 8)	All (parallel, ~half day)
5	Face similarity + threshold tuning against test pairs — swap into the already-integrated pipeline, schema unchanged	Module 4 (complete)	Person C
5-6	Two-axis risk scoring engine combining Modules 2, 3, 4 (real outputs now, same schema mock version used)	Risk engine (real)	Person A + B
6	Hash chain + embedding similarity search across ledger (real)	Module 6 (real)	Person C
6-7	Dashboard: upload, extracted fields, flags, tamper heatmap, face score, risk band	Frontend	All
7	Full integration, deploy, fix breakages	Integration	All
7-8 (if ahead of schedule)	Authority reference match using Mock DB #2	Module 5 (extension)	Whoever is free first
8	Test full fixture set against Section 6 matrix, tune thresholds	Testing	All
8-9	PPT, architecture diagram, Q&A rehearsal (Sections 6 & 7 especially), backup demo video	Presentation	All
9	Buffer + final rehearsal	—	All

If time runs short: Module 5 is the first thing to drop — but still present its design (Section 4) as a stated extension, not silence. Everything else in this plan is core and should not be cut.

9a. Mock-First Integration Strategy — Why Day 2-3, Not Day 7

This is the single highest-leverage practice in the whole plan for a small team on a tight deadline: **integrate the full pipeline before any module's real logic is finished, using fake outputs that already match the final schema.**

Why this matters

Without this, the natural failure mode is: each person builds their module in isolation for a week, and only on Day 7 does anyone actually run Module 1's real output into Module 2, or Module 4's real output into the risk engine. That's when you discover a field is named differently than expected, a date is a string when the next module expects a `datetime`, or Module 4's embedding is a different length than what Module 6 expects to store — and you have almost no time left to fix it. This is exactly the class of failure Section 13 lists, and mock-first integration prevents it structurally rather than relying on everyone remembering the schema correctly by hand.

How to run it

1. **Day 1:** the shared schema (Section 13) is agreed and written down — this must exist before any stub is written.
2. **Day 2:** every module owner writes a **stub function** — same function name and signature the real one will eventually have, but it just returns the exact standardized mock output defined in Section 9b, unmodified, no actual OCR/CV/ML code yet.

3. **Day 2-3:** whoever owns backend integration wires all the stubs together into the real FastAPI pipeline — upload endpoint calls stub Module 1 → stub Module 2 → stub Module 3 → stub Module 4 → stub risk engine → stub Module 6 → dashboard renders it. At the end of Day 3, you have a **fully working end-to-end demo with fake data** — this alone is a huge psychological and practical milestone this early in an 8-9 day sprint.
4. **Day 3 onward:** each module owner replaces their stub with real logic **in place** — same function signature, same output schema, just real computation inside. Because the schema was fixed on Day 1 and never changed, swapping a stub for the real implementation should require zero changes anywhere else in the pipeline.
5. **Any schema change discovered mid-build** (e.g. “we actually need a `no_face_detected` status, not just true/false”) gets made once, in the shared schema doc, and communicated to everyone same-day — not silently handled inside one module’s code.

What this buys you

- Integration bugs surface on Day 2-3, when you have 6+ days to fix them, instead of Day 7, when you have almost none.
- Every module can be developed and tested independently against a known, stable contract — nobody is blocked waiting for someone else’s module to be “done enough” to test against.
- You always have a demoable pipeline, even if some modules still have mocked logic — useful for early rehearsal and for showing steady progress if anyone checks in on the team.

9b. Standardized Mock Output — Exact Stub Return Value Per Module

Every module owner copies their block below **verbatim** into their stub function on Day 2. This is the literal, final contract each module must produce — real code written from Day 3 onward must return data in this exact shape, just with computed values instead of hardcoded ones. Nobody should need to guess field names, types, or ranges from prose description alone.

Module 1 — OCR Extraction

[illegible]

- `status` must always be one of: `"success"`, `"partial"` (some fields unreadable), `"failed"` (no usable text found) — never throw an exception for a bad image, return `"failed"` instead.
- `confidence` is always `0.0 - 1.0`, per field, no exceptions — this is required by the risk engine's evidence-strength axis (Section 5).
- Dates are always `YYYY-MM-DD` (ISO 8601) — non-negotiable, this exact failure is listed in Section 13.
- For visa/other doc types, `extracted_fields` keys change per the Section 11 config (e.g. visa uses `visa_number`, `visa_type`, `entry_validation`, `stay_duration`) but the `{ "value": ..., "confidence": ... }` wrapper shape stays identical for every field, every doc type.

Module 2 – Document Validation

```

{
  "module": "document_validation",
  "status": "success",
  "checksum_pass": true,
  "text_mrz_match": true,
  "expiry_valid": true,
  "db_status": "clear",
  "flags": []
}

```

- `db_status` is always one of: `"clear"`, `"blacklisted"`, `"expired"`, `"not_found"`.
- `flags` is a list of short strings when something fails, e.g. `["checksum_fail", "expiry_passed"]` — empty list when clean. The dashboard reads this list directly to display flags; don't invent a different failure-reporting shape later.
- `checksum_pass` / `text_mrz_match` are `null` (not `true` / `false`) when `mrz_format` is `"none"` for that doc type (Section 11) — this must be handled explicitly, not defaulted to `true`.

Module 3 — Tampering Detection

```

{
  "module": "tampering_detection",
  "status": "success",
  "ela_score": 0.12,
  "ela_clarity": 0.88,
  "copy_move_flag": false,
  "exif_flag": false,
  "exif_details": {"editing_software_detected": null, "modified_after_creation": false},
  "overall_tamper_flag": false
}

```

- `ela_score` and `ela_clarity` are always `0.0 - 1.0` — score is how much error was found, clarity is how trustworthy that reading is (Section 5's evidence-strength axis reads `ela_clarity` directly).
- `overall_tamper_flag` is the single boolean the risk engine actually consumes — it's this module's job to combine ELA/copy-move/EXIF into that one flag, not the risk engine's.

Module 4 — Face Verification

```

{
  "module": "face_verification",
  "status": "match",
  "similarity": 0.87,
  "threshold_used": 0.6,
  "face_detected_in_document": true,
  "face_detected_in_live": true
}

```

- `status` is always one of: `"match"`, `"mismatch"`, `"no_face_detected"` — this exact enum is referenced in Section 13's failure-case table, don't rename it.
- `similarity` is `null` when `status` is `"no_face_detected"` — never a fake number.

Module 5 — Authority Reference Match (extension)

```

{
  "module": "authority_reference_match",
  "status": "not_available",
  "similarity": null,
  "reference_source": "mock_db_2"
}

```

- Every other module must run correctly even when Module 5 returns `"not_available"` — this is what lets the whole pipeline work in the days before Module 5 exists, and still work if it's cut entirely per Section 9's fallback plan.
- Once built, `status` becomes one of: `"match"`, `"mismatch"`, `"not_available"` (no authority record found for that ID) — same three-state pattern as Module 4, deliberately, so the dashboard can reuse one rendering component for both.

Module 6 — Blockchain Ledger

```

{
  "module": "blockchain_ledger",
  "status": "success",
  "record_hash": "9f86d081884c7d659a2feaa0c55ad015a3bf4f1b2b0b822cd15d6c15b0f00a08",
  "previous_hash": "e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b85",
  "timestamp": "2026-09-01T10:15:00Z",
  "identity_reuse_flag": false,
  "identity_reuse_matches": []
}

```

- `identity_reuse_matches` is a list of prior record IDs when `identity_reuse_flag` is `true` (Module 6b’s embedding-search result) — empty list otherwise.

Risk Scoring Engine (not a numbered module, but needs the same stub treatment)

```

{
  "risk_score": 12,
  "risk_band": "clear",
  "evidence_strength": 0.91,
  "evidence_band": "high",
  "final_recommendation": "clear",
  "contributing_flags": []
}

```

- `risk_band` is always one of: `"clear"`, `"secondary_check"`, `"high_risk"` (Section 5’s three bands).
- `evidence_band` is always one of: `"high"`, `"low"`.
- `final_recommendation` is the single field the officer dashboard displays most prominently — combines both axes per the Section 5 matrix, so the dashboard never has to compute this logic itself, only display it.

10. Do’s and Don’ts

Do: - Get a thin end-to-end pipeline working with **mocked outputs by Day 2-3** (Section 9a), then swap in real logic module by module - Build tampered + clean-single-save fixtures yourselves — no dataset provides these - Present Module 5 and Section 7 even if Module 5 isn’t fully built — this is what shows depth - Cite real sources: ICAO Doc 9303 (MRZ), Krawetz 2007 (ELA), Lowe 2004 (SIFT), Schroff et al. 2015 (FaceNet) - Keep a backup recorded demo video

Don’t: - Don’t claim the system detects all forgery — claim it triages and automates the digitally-verifiable majority, with Module 5 named as closing the biggest remaining gap - Don’t conflate the hash chain (Module 6a) with fraud detection (Module 6b) — they solve different problems - Don’t skip the evidence-strength axis — a blurry “clean” pass is a false sense of security - Don’t present mock databases apologetically — frame them per Section 7: same logic, different data source

11. Multi-Document-Type Handling — Config-Driven Strategy

The problem statement lists 5 input types, but only Passport and Visa have explicitly defined fields. National ID, Driving License, and Permit are named but undefined — treat these as secondary, generic-OCR-only document types.

Per-module effect of document-type variety

Module	Affected by doc type?	Why	How to handle
Module 1 (OCR)	Heavily	Field layout, presence/format of MRZ, and expected fields differ completely per doc type (passport TD3 MRZ ≠ ID card TD1 MRZ ≠ driving license with no MRZ at all)	Config-driven strategy dispatch (below)
Module 2 (Validation)	Heavily	Checksum only applies where MRZ exists; field rules differ (e.g. “Stay Duration” only exists for visas); mock DB schema must be doc-type aware	Config-driven strategy dispatch (below)

Module 3 (Tampering Detection)	Minimal	ELA/EXIF/copy-move operate on pixel data and compression history — this is true regardless of what the document semantically is	No dispatch needed; same algorithm runs on any image
Module 4 (Face Verification)	Minimal	Face detection works on the whole image regardless of document type, as long as a face photo exists somewhere on it	No dispatch needed; face detector doesn't care about doc type
Module 5 (Authority Reference)	Minor	Just needs the correct ID field (passport no. / visa no. / ID no.) as the DB lookup key	Reuses whichever ID field Module 1 extracted for that doc type
Module 6 (Blockchain)	None	Logs whatever structured data it receives, doc-type agnostic	No dispatch needed

Key takeaway: the doc-type complexity is concentrated almost entirely in Modules 1 and 2. Modules 3, 4, 5, 6 should be built once and reused unmodified across every document type — don't let doc-type handling leak into them.

Config-driven dispatch design

For the hackathon, **have the officer/user select the document type from a dropdown at upload** rather than building an auto-classifier — auto-detecting document type is a separate, non-trivial ML problem and is explicitly out of scope for your 8-9 day window. State this as a deliberate scope decision, not an oversight.

Maintain a single config file (`doc_types_config.json`) as the source of truth:

```
{
  "passport": {
    "mrz_format": "TD3",
    "checksum_required": true,
    "expected_fields": ["name", "passport_number", "nationality", "dob", "expiry", "gender"],
    "validation_rules": ["checksum", "expiry_check", "text_mrz_crosscheck"]
  },
  "visa": {
    "mrz_format": "TD3_or_none",
    "checksum_required": false,
    "expected_fields": ["visa_number", "visa_type", "entry_validation", "stay_duration"],
    "validation_rules": ["expiry_check", "stay_duration_range_check"]
  },
  "national_id": {
    "mrz_format": "TD1",
    "checksum_required": true,
    "expected_fields": ["name", "id_number", "dob"],
    "validation_rules": ["checksum"]
  },
  "driving_license": {
    "mrz_format": "none",
    "checksum_required": false,
    "expected_fields": ["name", "license_number", "dob", "expiry"],
    "validation_rules": ["expiry_check"]
  },
  "permit": {
    "mrz_format": "none",
    "checksum_required": false,
    "expected_fields": ["name", "permit_number", "validity"],
    "validation_rules": ["expiry_check"]
  }
}
```

A single dispatcher function reads the selected `doc_type`, loads its config, and calls the matching OCR/validation strategy — e.g. `run_ocr(image, config)` internally branches to `parse_mrz()` only if `mrz_format != "none"`, otherwise falls straight to generic field-region OCR. This keeps Module 1/2 code as one shared pipeline with per-type behavior driven entirely by config, not scattered if/else logic — much easier to demo and explain, and easy to extend to a 6th document type later by just adding a config entry.

12. Data & Mock DB Task Allocation — By Skill Level

Task	Best suited to	Why	Guidelines
------	----------------	-----	------------

Fixture creation (GIMP edits: photo swap, digit alter, stamp clone)	A less-experienced or detail-oriented teammate	Requires care and attention, not deep algorithmic skill — a genuinely useful, confidence-building contribution	Keep a manifest file logging every fixture: filename, doc type, what was altered, expected module 3 result (tampered/clean)
Single-save clean doc generation (Python + Pillow)	Whoever is comfortable with basic Python, doesn't need to be your strongest coder	Simple scripted image composition, no CV/ML knowledge needed	Reuse one script/template so all clean fixtures are generated consistently
Mock DB #1 (issuance/blacklist) schema + population	Backend-comfortable teammate	Structured data + simple lookup functions	Use the exact same ID values that appear in your fixture images — a single typo here silently breaks Module 2 tests
Mock DB #2 (authority reference photos)	Same person as Mock DB #1, for consistency	Same lookup-table pattern	Map passport number → the <i>original unedited</i> MIDV-500 photo, before your team's edits
Face pair curation (selfies, match/mismatch labeling)	Whole team (each member provides their own selfie)	Needs real people, quick task	Store as <code>matched/</code> and <code>mismatched/</code> folders, named by teammate, so nobody has to guess later which pair is which

Golden rule for every fixture: maintain one central `fixtures_manifest.csv` (or JSON) with columns: `filename`, `doc_type`, `category` (clean/tampered/authority_ref), `expected_module3_result`, `expected_module4_result`, `linked_db_id`. Without this, by Day 7 nobody remembers which of 30 images was supposed to do what — this single file prevents that entirely and doubles as your test suite's ground truth.

13. Module I/O Contracts — Avoiding Integration and Explainability Issues

Define these on **Day 1**, before any module is built, and don't deviate without updating the shared doc.

Common failure cases and how to prevent them

Failure case	Which modules it breaks	Prevention
Date format mismatch (Module 1 outputs <code>"04/05/88"</code> , Module 2 expects ISO <code>"1988-05-04"</code>)	1 → 2	Standardize on ISO 8601 for every date field, everywhere, from Day 1 — write this rule down and share it
Field name mismatch (<code>"dob"</code> vs <code>"date_of_birth"</code> vs <code>"DOB"</code>)	1 → 2, 1 → 5	Agree on one exact field-name schema (see below) before writing any code, not after
Missing confidence score (Module 1 returns text only, no per-field confidence)	1 → risk engine	Require every Module 1 output to include a <code>confidence</code> value per field from the start — retrofitting this later is expensive
OCR/face-detection failure not handled (module throws an exception instead of returning a status)	1, 4 → whole pipeline crashes	Every module must return a defined "failure" state (e.g. <code>{"status": "no_face_detected"}</code>) — never let an unhandled exception break the pipeline mid-demo
Embedding dimension mismatch (Module 4 uses one face model, Module 5/6 assume another)	4 → 5, 4 → 6	Lock one face-embedding model/library for the entire project on Day 1 — do not let different teammates pick different libraries independently
Thresholds hardcoded differently in different files (face-match threshold set to 0.6 in Module 4's code, 0.65 in the risk engine)	4 → risk engine	Keep all thresholds in one shared config file, referenced everywhere — never duplicate a magic number
Mock DB ID mismatch (fixture image says passport number <code>x</code> , DB entry says <code>x1</code> due to a typo)	2, 5	Enforced by the fixtures manifest (Section 12) — cross-check IDs before integration, not during
Blockchain ledger race condition (two screenings write near-simultaneously, corrupting the hash chain)	6	For the demo, process screenings sequentially (no need for concurrency handling) — explicitly simplify this and say so if asked

Minimum shared schema (agree on this before Day 2)

```

{
  "doc_type": "passport",
  "extracted_fields": {
    "name": {"value": "...", "confidence": 0.0-1.0},
    "passport_number": {"value": "...", "confidence": 0.0-1.0},
    "dob": {"value": "YYYY-MM-DD", "confidence": 0.0-1.0},
    "expiry": {"value": "YYYY-MM-DD", "confidence": 0.0-1.0}
  },
  "validation": {"checksum_pass": true, "db_status": "clear|blacklisted|expired|not_found"},
  "tamper": {"ela_score": 0.0-1.0, "ela_clarity": 0.0-1.0, "copy_move_flag": true, "exif_flag": true},
  "face": {"similarity": 0.0-1.0, "status": "match|mismatch|no_face_detected"},
  "authority_match": {"similarity": 0.0-1.0, "status": "match|mismatch|not_available"}
}

```

Every module reads/writes only its own section — the risk engine is the only piece allowed to read all sections together. This single contract, agreed early, is what prevents Day 7 integration from turning into a debugging scramble.

For the exact per-module stub return values (what each module’s Day 2 mock function must literally output), see Section 9b — that is the authoritative, copy-pasteable version of this schema.

14. Team Coordination Guidelines

Team lead responsibilities: - Own the sprint plan (Section 9) and check progress against it every day, not just at milestones - Own the shared schema (Section 13) — no teammate changes it unilaterally - Own the demo script and rehearsal schedule - Do a daily 10-minute sync: what’s done, what’s blocked, does anything affect another module’s contract

Pairing strategy: pair your stronger and less-experienced teammates on the *same* module rather than splitting modules purely by perceived skill level — e.g., stronger person builds the ELA/SIFT logic, less-experienced person builds and documents the test fixtures for that exact module and verifies results against the manifest. This means the less-experienced teammate deeply understands one module end-to-end, rather than being a passive helper across everything.

For Q&A and demo readiness — every teammate should own one explainable piece: - Judges commonly question each team member individually — a teammate who can only say “I don’t know, they built that” hurts the whole team’s credibility - Assign each teammate 1-2 modules they are personally responsible for being able to explain, even if they didn’t write all the code for it — prioritize giving your least experienced teammate the most conceptually simple parts to own: fixture design/manifest (Section 12), or the blockchain hash-chain narrative (Section 4, Module 6a — it’s a clean, self-contained concept to explain) - Run at least 2 full mock Q&A rounds before submission day, rotating who answers which question

Demo script — rehearse this exact sequence, mapped to your fixtures manifest: 1. Clear document, full pass (all green) 2. Digitally tampered document → Module 3 catches it 3. Photo swapped to a different, wrong person → Module 4 catches it 4. Blacklisted/expired document → Module 2 catches it 5. (If Module 5 built) Flawless-forgery-style fixture → Module 4 passes, Module 5 catches it — this is your strongest differentiation moment, don’t skip it if built 6. Show the blockchain ledger and the dashboard’s risk/evidence-strength bands

Backup plan: have a recorded video of this exact sequence ready, in case live demo/network fails.

15. Overall Problem Statement Coverage — What’s Addressed, What’s Not

Problem statement item	Addressed?	How / Why not
Fake passports and visas	Mostly	Digital forgery: yes (Module 3). Physical forgery with correct checksum/MRZ: only if Module 5 is built; otherwise honest gap (Section 6/7)
Altered photographs	Yes	Module 3 (edit artifacts) + Module 4 (wrong-person mismatch) + Module 5 (authority reference, if built)
Modified dates of birth	Yes	Module 2 checksum + text/MRZ cross-check
Tampered visa stamps	Partially	Module 3’s copy-move detection catches

		duplicated/cloned stamps; a well-executed new stamp with no cloning artifact is not reliably caught by digital forensics alone
Identity impersonation	Yes	Module 4 (and Module 5, if built, for the flawless-forgery case)
Multiple identities used by the same person	Yes	Module 6b — embedding similarity search across the ledger
Expired or blacklisted travel documents	Yes	Module 2 + Mock DB #1
High passenger volume causing delays	Yes	Full automated pipeline runs in seconds; explicitly a triage system, not full replacement of manual checks (Section 1)
OCR Extraction (Module 1)	Yes	Passport/visa fully spec'd; National ID/DL/permit generic OCR, secondary priority
Document Validation (Module 2)	Yes	Rule/checksum/DB-based, as designed
Tampering Detection (Module 3)	Yes, with a stated limit	Strong on digital tampering; physical-only forgery (no digital edit, no cloning, correct materials) is outside what image forensics alone can catch — stated explicitly, not hidden
Face Verification (Module 4)	Yes, with a stated limit	Catches wrong-person impersonation; flawless-forgery-with-own-face case needs Module 5
Reduce verification time to seconds	Yes	Core design goal — automated pipeline vs. manual reading
Improve detection of forged/tampered documents	Yes, with stated ceiling	Meaningfully improves over pure manual review; does not claim to catch every forgery type (see Section 6/7)
Standardize screening decisions across checkpoints	Yes	Rule-based validation applies identical logic regardless of officer/shift
Data-driven risk assessment	Yes	Two-axis risk scoring engine (Section 5)
Digital trail for investigations	Yes	Module 6 — blockchain ledger
Blockchain & Cybersecurity theme	Yes	Module 6 directly; also the theme's implicit "security honesty" is reflected in Section 7's prototype-vs-real distinctions

Where it doesn't fully address the problem statement — say this plainly if asked, don't let it be discovered by a question:

- **Physical-only forgery** (no digital editing, materials/print quality good enough to fool visible-light image forensics, no cloned regions) is not reliably caught by Modules 1-4 alone. Module 5 closes the specific flawless-forgery-with-matching-face sub-case, but general high-skill physical forgery still needs hardware (UV/IR/hologram) that is out of scope for this prototype (Section 7).
- **Auto-detection of document type** is not built — the officer selects it manually (Section 11). This is a deliberate scope decision to keep the project buildable in 8-9 days, not a missed requirement — the problem statement doesn't explicitly require automatic type detection.
- **Driving license and permit fields** are not explicitly defined by the problem statement, so these are handled with generic OCR only, at lower confidence than passport/visa — stated as a secondary-priority category by design.